

Von Knicken zu Gedanken

Wie moderne KI-Systeme wirklich funktionieren

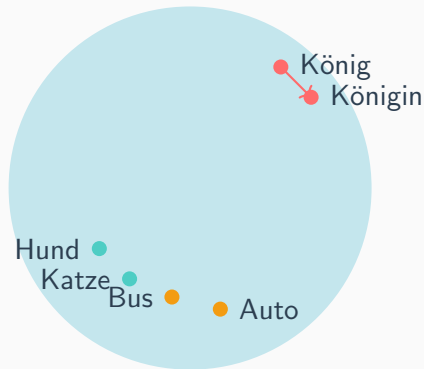
Folgevortrag – Aufbauend auf: *Was sind Embeddings?*

2026

Kurzer Rückblick

Was wir letztes Mal gelernt haben

- Wörter = Punkte im Raum
- Ähnliche Bedeutung = nah beieinander
- $\text{König} - \text{Mann} + \text{Frau} \approx \text{Königin}$
- Hunderte Dimensionen

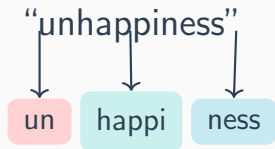


Heute: Was passiert mit diesen Punkten *innerhalb* des Netzwerks?

Tokenization

Schritt 0: Tokenization – Wie liest die KI?

Die KI liest nicht in **Wörtern**, sondern in **Tokens**!



Überraschung!

„Strawberry“ → [St, raw, berry] – Das Modell *sieht* die einzelnen Buchstaben gar nicht direkt!

Warum Tokens statt Wörter?

Problem mit ganzen Wörtern:

- Zu viele verschiedene Wörter
- Neue Wörter? Unbekannt!
- „Donaudampfschiffahrt“ = ???

Problem mit einzelnen Buchstaben:

- Sätze werden extrem lang
- Zusammenhänge schwer zu lernen

Tokens = Goldener Mittelweg

- Ca. 50.000–100.000 Tokens
- Häufige Wörter = 1 Token
- Seltene Wörter = mehrere Tokens
- „the“ = 1 Token
- „Do-
nau“ + „dampf“ + „schiff“ + „fahrt“

Jedes Token bekommt einen Embedding-V

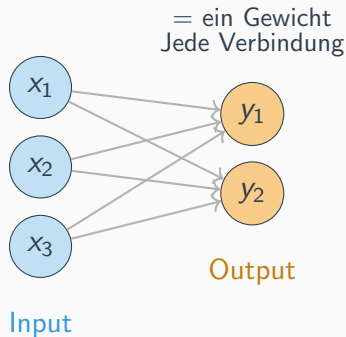
Dense Layer – Universelle Approximation

Was ist ein Dense Layer?

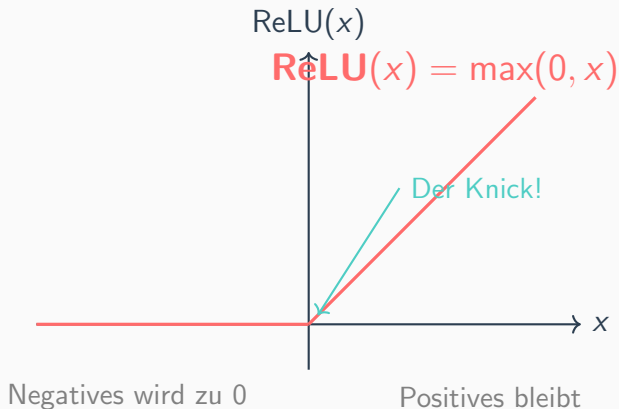
Einfachste Recheneinheit:

$$\text{output} = W \cdot \text{input} + b$$

- W = Gewichte (werden gelernt)
- b = Bias (Verschiebung)
- Das ist nur eine **lineare Funktion!**
- Alleine: kann nur Geraden

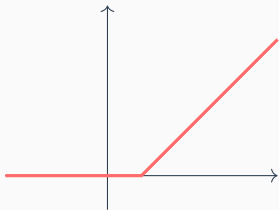


Das Geheimnis: ReLU – Der Knick

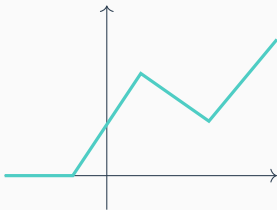


Klingt simpel? **Ist es auch.** Aber daraus entsteht Magie!

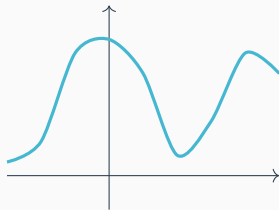
Viele Knicke = Jede Funktion!



1 ReLU = 1 Knick



3 ReLUs = 3 Knicke

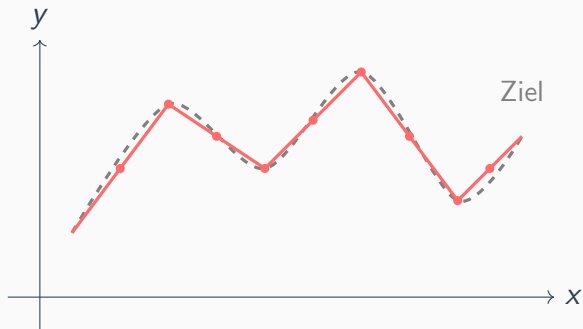


Viele ReLUs $\approx$ alles!

Universal Approximation Theorem

Ein Netzwerk mit **Dense + ReLU + Dense** kann *jede* kontinuierliche Funktion beliebig genau approximieren – wenn es genug Neuronen hat!

Intuition: Warum funktioniert das?



Jeder Punkt = ein ReLU-Neuron = ein Knick

Mehr Parameter $\rightarrow$ mehr Knicke $\rightarrow$ genauere Approximation

GPT-4: $\sim 1.800.000.000.000$ Parameter = **sehr viele Knicke!**

Der Transformer – Überblick

Der Transformer: Das Gesamtbild

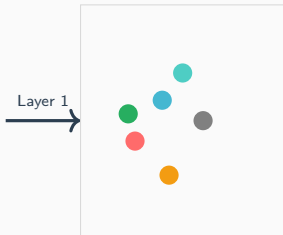


Was passiert mit den Token-Punkten?

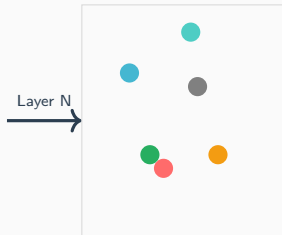
Eingang



Nach Layer 1



Nach Layer N



Jeder Layer **verschiebt** die Punkte im Raum.

Am Ende kodiert jeder Punkt nicht mehr nur das Wort selbst, sondern den **gesamten Kontext!**

Attention

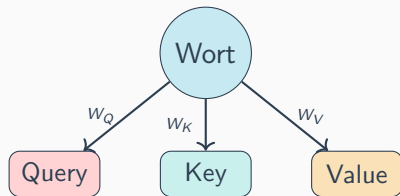
Attention: Wer schaut wen an?



Attention = Gewichteter Blick

Jedes Wort „entscheidet“, wie viel Aufmerksamkeit es jedem anderen Wort schenkt. So fließt Information zwischen Wörtern!

Attention: Die Mechanik



Was suche ich? Was biete ich an? Was ist mein Inhalt?

Attention(Q, K, V) =

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

Query · Key = Wie gut passt es?
Ergebnis gewichtet die Values

Attention ist wie eine Party!



Query

„Ich suche jemanden, der
mir bei Mathe helfen
kann“



Key

„Ich bin Mathematiker
und kenne mich mit
Zahlen aus“



Value

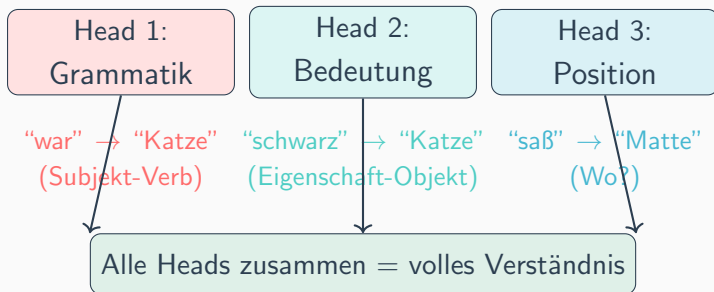
„Die Antwort ist 42, und
hier ist der Rechenweg...“

Query trifft auf passenden Key → Value fließt!

(Und das passiert **gleichzeitig** für alle Wörter im Satz!)

Multi-Head Attention: Verschiedene Aspekte

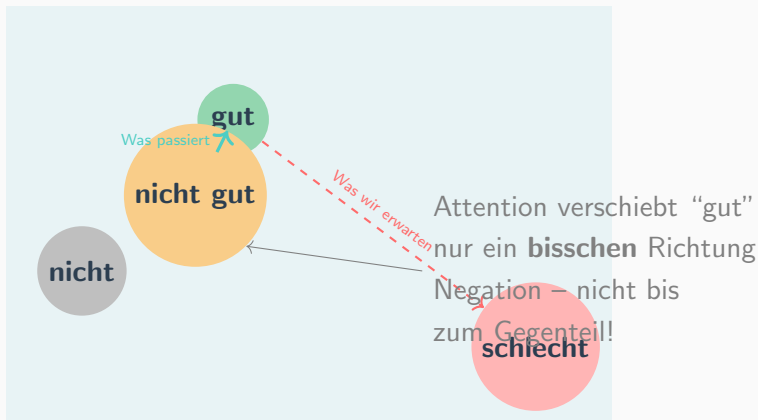
“Die Katze, die auf der Matte saß, war schwarz”



GPT-4 hat **128 Attention Heads** pro Layer – es schaut auf 128 verschiedene Aspekte gleichzeitig!

Das Problem mit “nicht”

Warum “nicht gut” $\neq$ “schlecht”



Aha!

“nicht” verschiebt den Punkt von “gut” nur leicht – er bleibt in der *Nähe* von “gut” | Deshalb verwechseln LLMs manchmal Negationen

Warum ist das so?

- **Attention** kann Punkte nur als **gewichtete Summe** anderer Punkte verschieben
- “nicht” + “gut” = irgendwo *zwischen* den beiden Punkten
- Es gibt kein “springe direkt zum Gegenteil”-Modul!
- Das Netzwerk muss über **mehrere Layer** hinweg lernen, Negation zu verarbeiten

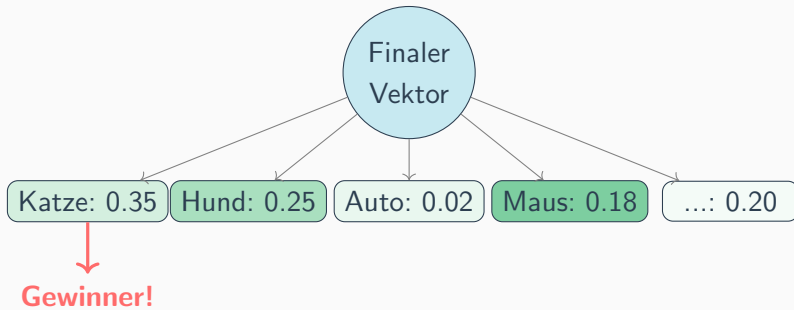
Das erklärt, warum LLMs manchmal bei
“Nenne mir etwas, das KEIN Tier ist”
trotzdem Tiere nennen!

Next-Token-Prediction

Das Finale: Welches Wort kommt als nächstes?

Input: "Die flauschige ___"

Vergleich mit ALLEN bekannten Tokens:



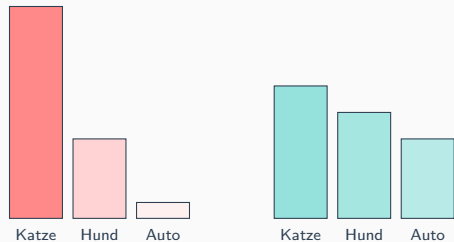
Sampling: Zufall als Feature

Wahrscheinlichkeiten:

- Katze: 35%
- Hund: 25%
- Maus: 18%
- Hamster: 12%
- Auto: 2%
- ...: 8%

Das Modell nimmt **nicht immer** das wahrscheinlichste Wort!

Temperature

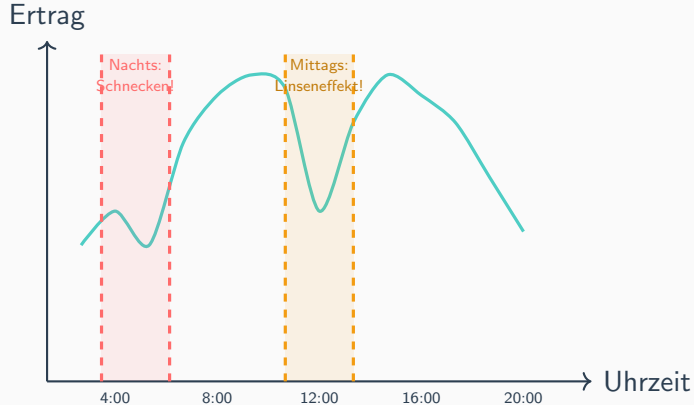


Temp=0.1: Vorhersagbar Temp=1.5: Kreativ

Temperature steuert, wie “mutig” das Modell bei der Wortwahl ist.

Praktisches Beispiel: Bewässerung

Praxis: Ein Netzwerk lernt Bewässerung



Das Netzwerk **lernt** die Dips – aber **versteht** es warum?

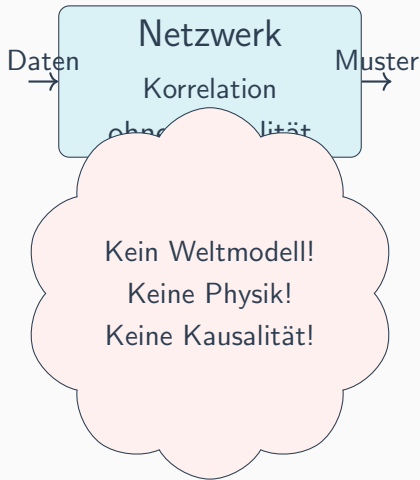
Lernen ohne Verstehen

Was das Netzwerk "weiß":

- Nachts wässern → schlechter Ertrag ✓
- Mittags wässern → schlechter Ertrag ✓
- Morgens/Abends → guter Ertrag ✓

Was es NICHT weiß:

- Warum Schnecken nachts kommen
- Wie Wassertropfen als Linse wirken
- Was passiert wenn man Schneckenzaun baut



Haben große Sprachmodelle ein **Weltmodell**?



Hinweise dafür:

- Weltkarte in Embeddings
- Logisches Schlussfolgern
- Modular Arithmetic
- Konsistente Persönlichkeit



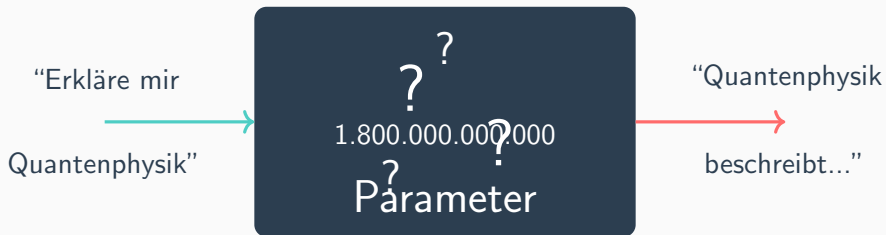
Hinweise dagegen:

- Halluzinationen
- Zählfehler (Strawberry)
- Logikfehler bei Negation
- Kein echtes Verständnis von Zeit

Die Wahrheit liegt vermutlich irgendwo dazwischen.

Die Black Box

Das Problem: Milliarden Parameter



Niemand versteht vollständig, **was in der Box passiert.**

Wir wissen wie die Architektur aussieht – aber nicht, was die gelernten Gewichte *bedeuten.*

Mechanistic Interpretability

Ein junges Forschungsfeld versucht, die Box zu öffnen

Ziel:

- Verstehen was *einzelne* Neuronen tun
- Schaltkreise (Circuits) finden
- Algorithmen identifizieren die das Netzwerk gelernt hat

Methoden:

- Activation Patching
- Probing Classifiers
- Sparse Autoencoders
- Ablation Studies

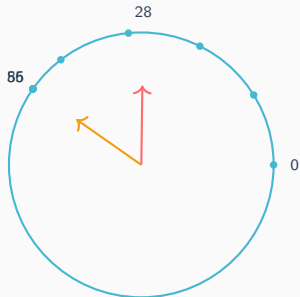
Hauptakteure: Anthropic, DeepMind, Neel Nanda, Chris Olah

Entdeckung: Modular Addition (Neel Nanda, 2023)

Aufgabe: Trainiere ein kleines Netzwerk auf $(a + b) \bmod 113$

Was das Netzwerk lernt:

1. Zahlen als **Punkte auf einem Kreis** darstellen (Fourier!)
2. Addition = **Rotation** auf dem Kreis
3. Modulo = der Kreis wiederholt sich

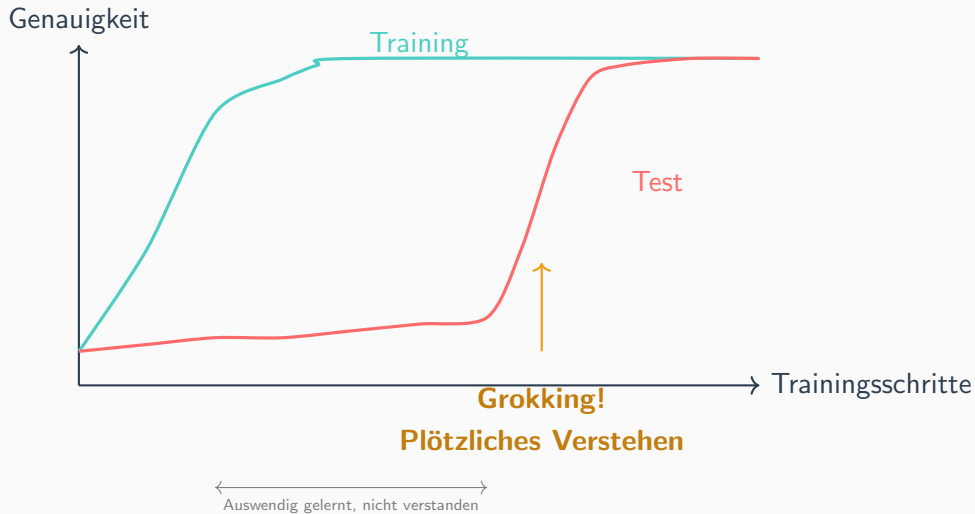


Zahlen auf einem Kreis

Aha!

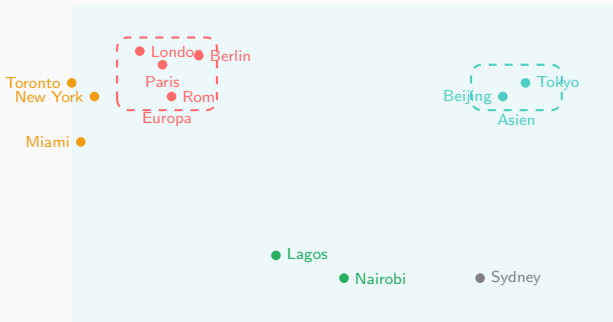
Niemand hat dem Netzwerk

Grokking: Plötzliches Verstehen



Weltkarte aus Embeddings

Städte-Embeddings $\approx$ echte Weltkarte!



Gurnee & Tegmark, 2023

Die internen Repräsentationen von Städtenamen bilden **automatisch** eine Karte
– weil das die effizienteste Art ist, relative Entfernungen zu speichern!

Warum entsteht die Karte?

“Paris liegt westlich von Berlin”
“Tokyo ist weit von London”
“Rom liegt südlich von Berlin”
“New York liegt westlich von London”
...

Training

Effizienteste Kodierung
aller relativen Beziehungen
= **echte Geographie!**

Das Modell hat **nie eine Karte gesehen** –
es hat die Struktur der Welt aus **Text allein** rekonstruiert!

(Ähnlich: Zeitliche Abfolgen, Farbräume, Verwandtschaftsbeziehungen...)

Explainability

Explainability: Verschiedene Ansätze

Post-hoc

Erklärung nach
der Vorhersage

SHAP, LIME, Attention Maps

Mechanistic

Interne Schaltkreise
verstehen

Circuits, Neel Nanda

Probing

Was ist wo
gespeichert?

Weltkarte, Syntax-Probes

Feature Viz

Was aktiviert
ein Neuron?

Sparse Autoencoders

Was wir bisher gefunden haben

- **Induction Heads:** Schaltkreise die Muster wie “A B ... A $\rightarrow$ B” erkennen
- **Syntax-Neuronen:** Einzelne Neuronen die auf Satzstrukturen reagieren
- **Fakten-Speicherung:** Bestimmte Layer speichern Faktenwissen (“Paris ist die Hauptstadt von...”)
- **Fourier-Algorithmen:** Netzwerke erfinden eigenständig mathematische Verfahren
- **Weltkarte:** Geographische Struktur emergiert aus reinem Text

Wir verstehen vielleicht **1–5%** dessen,
was in einem großen Sprachmodell passiert.

Aber es wird jedes Jahr mehr!

Theorien: Was passieren könnte

Theorie 1: Kompression

- LLMs komprimieren das Internet
- Verstehen = effiziente Kompression
- Muster erkennen = komprimieren

Theorie 2: Simulation

- LLM simuliert einen "Autor"
- Verschiedene Prompts = verschiedene simulierte Personen
- Kein echtes "Ich"

Theorie 3: Emergente Weltmodelle

- Ab gewisser Größe entstehen echte interne Modelle
- Weltkarte als Beweis?
- Vielleicht "Verstehen light"

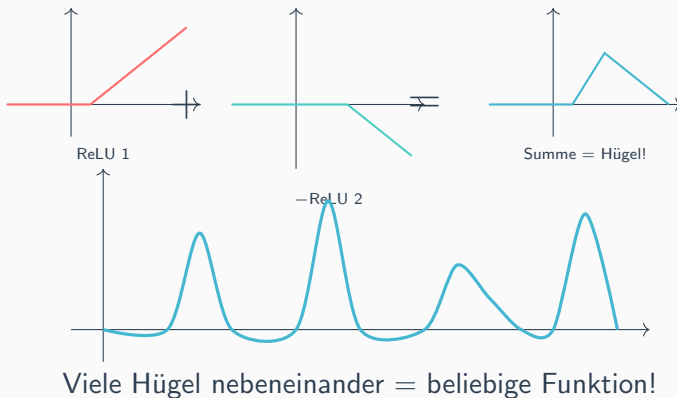
Theorie 4: Stochastischer Papagei

- Nur statistische Muster
- Kein echtes Verständnis
- Nur sehr gute Autokorrelation

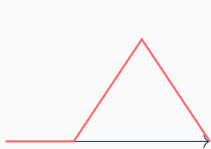
Dense kann alles approximieren

Nochmal: Warum Dense + ReLU so mächtig ist

Baukasten-Prinzip:



In höheren Dimensionen



1D: Knicke



2D: Flächen



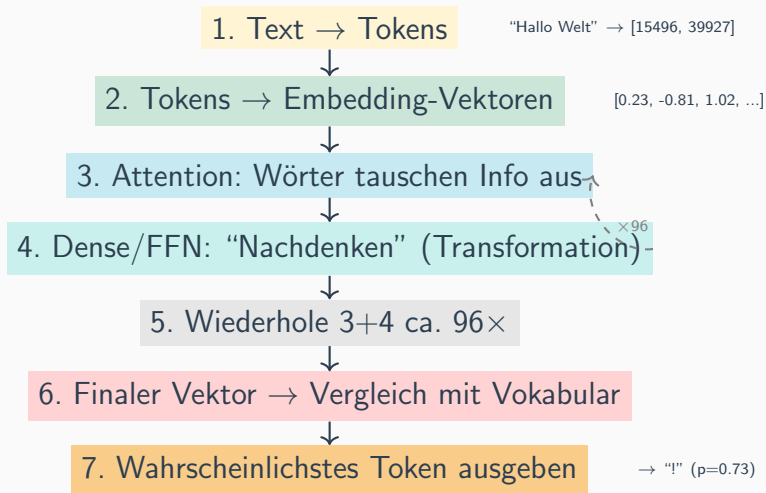
nD : Hyperebenen

Im Transformer

- Embeddings haben **4096+ Dimensionen**
- Dense Layers arbeiten in diesem hochdimensionalen Raum
- Sie können beliebig komplexe Entscheidungsgrenzen formen
- Jeder Dense Layer: "Wenn der Punkt *hier* ist, verschiebe ihn *dorthin*"

Zusammenfassung

Der komplette Weg eines Prompts



Was wir heute gelernt haben

1. **Tokenization:** KI liest in Stücken, nicht in Wörtern
2. **Dense + ReLU:** Kann mit genug Knicken *alles* approximieren
3. **Attention:** Lässt Wörter miteinander “reden”
4. **Transformer:** Schiebt Punkte im Raum herum, Layer für Layer
5. **Next-Token:** Am Ende wird mit allen Wörtern verglichen
6. **Black Box:** Wir verstehen noch wenig vom Innenleben
7. **Aber:** Es gibt faszinierende Entdeckungen (Weltkarte, Fourier...)

LLMs: Unglaublich mächtig,
erstaunlich einfach im Prinzip,
und noch voller Geheimnisse.

“Wir haben Maschinen gebaut, die besser schreiben
können als die meisten Menschen –
und wir verstehen nicht, wie sie es tun.”

Das ist gleichzeitig faszinierend und beunruhigend.

Quellen & Weiterführendes

- **Weltkarte:** Gurnee & Tegmark (2023): “Language Models Represent Space and Time” – arXiv:2310.02207
- **Grokking/Mod 113:** Neel Nanda et al. (2023): “Progress measures for grokking via mechanistic interpretability” – arXiv:2301.05217
- **Attention:** Vaswani et al. (2017): “Attention Is All You Need” – arXiv:1706.03762
- **Universal Approximation:** Cybenko (1989), Hornik (1991)
- **Mechanistic Interpretability:** Anthropic Research Blog, Neel Nanda’s YouTube
- **Induction Heads:** Olsson et al. (2022): “In-context Learning and Induction Heads” – arXiv:2209.11895

Danke!

Fragen?

